Projekt- und Qualitätsmanagement | PmQm

Zusammenfassung

INHALTSVERZEICHNIS

1. Projekt oder Produkt ... 3
2. Innovation ... 3
3. Qualität ... 3
   - 3.1. Datenqualität ... 4
4. Projektmanagement ... 4
   - 4.1. Wasserfallmodell ... 4
     - 4.1.1. Definition einer Phase ... 4
     - 4.1.2. Arbeitspakete ... 4
   - 4.2. HERMES ... 4
     - 4.2.1. Rollen ... 5
     - 4.2.2. Aufgaben ... 5
     - 4.2.3. Ergebnisse ... 5
     - 4.2.4. Sichten auf das Projekt ... 6
     - 4.2.5. Szenarien ... 6
     - 4.2.6. Entscheidungsprozess ... 6
     - 4.2.7. HERMES und SCRUM ... 6
     - 4.2.8. Vor- und Nachteile ... 7
   - 4.3. V-Modell ... 7
   - 4.4. PRINCE2 ... 7
     - 4.4.1. Vier integrierte Bausteine ... 7
     - 4.4.2. Sieben Grundprinzipien ... 8
     - 4.4.3. Sieben Themen ... 8
     - 4.4.4. Sieben Prozesse ... 8
     - 4.4.5. PRINCE2 Agile ... 9
   - 4.5. Gantt ... 10
5. Agile Produktentwicklung ... 10
   - 5.1. Arbeitspakete ... 11
   - 5.2. Warum Agile? ... 11
     - 5.2.1. Stacey-Matrix ... 11
     - 5.2.2. Warum scheitern IT Projekte? ... 11
     - 5.2.3. Warum werden agile Methoden verwendet ... 11
   - 5.3. SCRUM ... 12
     - 5.3.1. Zeremonien ... 12
     - 5.3.2. Artefakte ... 13
     - 5.3.3. Fundament ... 13
   - 5.4. User Story und MVP ... 13
     - 5.4.1. Theme, Epic und User stories ... 14
     - 5.4.2. MVP ... 14
   - 5.5. Was heisst Agile ... 15
     - 5.5.1. (1) CICD ... 15
     - 5.5.2. (2) Kundennutzen ... 16

5.5.3. (3) Zusammenarbeit . . . 16
5.5.4. (4) Inspect and Adapt . . . 16
5.6. Kanban . . . 16
5.6.1. Nutzen . . . 16
5.6.2. Prinzipien . . . 17
5.6.3. Praktiken . . . 17
5.6.4. Timebox vs Functionbox . . . 17
5.6.5. Kanban und Scrum . . . 17
5.7. Weitere agile Methoden . . . 17
5.7.1. XP . . . 17
5.7.2. Scaled Agile Framework / Agilität im Multiteam . . . 18
6. Rational Unified Process (RUP) . . . 19
6.1. Allgemein . . . 19
6.1.1. Vorteile . . . 19
6.2. Prinzipien . . . 19
6.3. Phasen . . . 20
6.4. Iterationen - vier Regeln . . . 21
6.5. Disziplinen . . . 21
6.6. Iterative Projektplanung . . . 23
6.7. Typische Fehler . . . 23
7. IT-Servicemanagement . . . 23
7.1. Ziele . . . 23
7.2. Business Nutzen . . . 24
7.3. Vier Service Management Dimensionen . . . 24
7.4. Service Value System . . . 24

WARN:
This document is still under development and will be completed around 1.2.2027

1. PROJEKT ODER PRODUKT

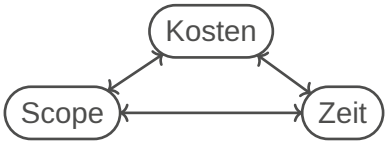
Produkt: ist etwas, das auf dem Markt verkauft werden kann

Projekt: eine Methode, die als Resultat ein Produkt hervorbringt

Projekt	*Produkt*
Einmaliges Vorhaben	Andauernde Entwicklung
Grosser Initialaufwand bis zum ersten sichtbaren Ergebnis	Mit MVP (Minimum Viable Product) sehr schnell ein erstes Ergebnis
Kundennutzen oft erst am Ende	Schneller und wachsender Kundennutzen
Mit OTOBOS sehr gute Planbarkeit von Zeit, Budget und Scope	Durch agile Entwicklungszyklen ständig ändernde Situation

OTOBOS: On Time, On Budget, On Specification

Das magische Dreieck:



2. INNOVATION

Innovation ist der Prozess, bei dem eine Idee in ein Produkt umgewandelt wird, und einen Mehrwert für den Kunden schafft

Inkrementelle Innovation: bedeutet die allmähliche, aber kontinuierliche Verbesserung bestehender Technologien (Markt für Smartphones)

Disruptive Innovationen: führen zu neuen Technologien (Musik und Film-Streaming, digitale Fotographie)

Architektonische Innovationen: passen bestehende Komponenten eines Produkts einen neuen Markt und Zweck an (Smart Watch, EarPod als Hörgerät)

Radikale Innovation: ersetzt bestehende Angebote vollständig und erschliesst einen neuen Markt (Cloud Computing, AI)

3. QUALITÄT

Software Qualität Komponenten

Funktionalität	Was kann die Software
Zuverlässigkeit	in einem vorbestimmten Zeitraum unter vorbestimmten Bedingungen immer gleich stabil arbeiten
Effizienz	gute Performance mit minimalen Ressourcenanforderungen Effizienz
Benutzbarkeit / Usability	Wie komfortable und fehlerresistent ist die SW für den Benutzer

Übertragbarkeit	Wie einfach ist die SW auf andere Plattformen / OS übertragbar
Änderbarkeit	Wie einfach können Anpassungen vorgenommen oder neue Releases eingespielt werden

3.1. DATENQUALITÄT

Jede Applikation und jedes Projekt ist maximal so gut, wie die Qualität der darin verwalteten Daten. Zu vermeiden sind:

– Unzuverlässige und fehlerhafte Daten
– Inkonsistente und duplizierte Daten

4. PROJEKTMANAGEMENT

Es wird zwischen agil und klassisch unterschieden
– Klassische Vorgehensmethoden sind formell und strikt, was klare Aussagen zu Fortschritt und Inhalt zulässt. Sie haben einen grossen administrativer Aufwand und zeigen erst am Schluss den Erfolg.
– Agile Vorgehensmethoden basieren dank iterativem Vorgehen auf schlanken Prozessen und eignen sich idealerweise für die Entwicklung und Pflege von Produkten. Sie reduzieren die administrativen Aufwände auf ein Minimum.

4.1. WASSERFALLMODELL

– Beim Wasserfallmodell werden die einzelnen Phasen sequenziell durchlaufen.
– Die nachfolgende Phase kann erst mit vollständigem Abschluss der aktuellen Phase beginnen.
– Phasenwiederholungen und -rücksprünge sind eigentlich nicht vorgesehen.
– Sie können aber notwendig sein, wenn eine Phase nicht planmässig erfolgreich abgeschlossen werden kann.

4.1.1. Definition einer Phase

Im klassischen Projektmanagement werden die Arbeitspakete oder Tasks in logische Abschnitte, sprich Phasen unterteilt. Jede Phase wird durch einen Meilenstein terminiert.

Phasenübergänge werden auch «Gates» genannt. Sie werden oft durch ein übergeordnetes Gremium (z.B. Lenkungsausschuss) freigegeben.

Eine Parallelisierung von Phasen ist nicht vorgesehen, kommt jedoch in der Praxis vor.

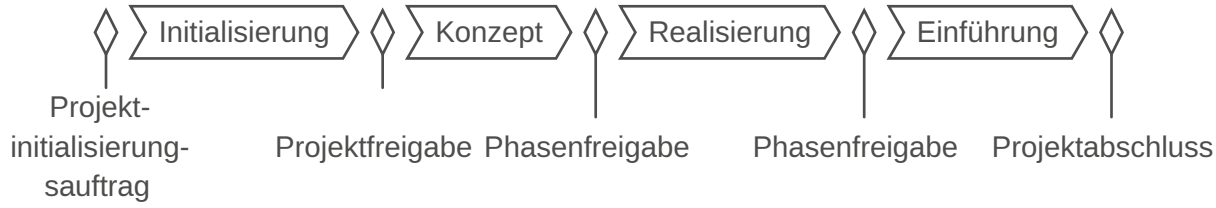
4.1.2. Arbeitspakete

Die Aufgaben in einem Projekt werden in Arbeitspakete gepackt. Jedes Arbeitspaket soll in sich abgeschlossen sein und ein definiertes Ergebnis liefern. Es soll wenn möglich von einer Person / Team erledigt werden können. Arbeitspakete sollen den Projektphasen zugeteilt werden können.

4.2. HERMES

Handbuch der Elektronischen Rechenzentren des Bundes, eine Methode zur Entwicklung von Systemen. HERMES beschreibt verschiedene Szenarien:

– Alle Szenarien haben ein einheitliches Phasenmodell mit vier Phasen.



– Die Meilensteine entsprechen Quality Gates, an denen über Ergebnisse und das weitere Vorgehen entschieden wird.
– Module sind wiederverwendbare Bausteine zur Erstellung von Szenarien.
– Ein Modul enthält die thematisch zusammengehörenden Aufgaben, Ergebnisse und Rollen.

4.2.1. Rollen

HERMES unterteilt die Organisation in:

Stammesorganisation:

– Organisation des Auftraggebers und des Anwenders in der das Projekt angesiedelt ist
– Rechtliche Einheit, die Strategien und Vorgaben für das Projekt bestimmt

Projektorganisation:

– Temporäre Organisation
– Wird mit dem Projektinitialisierungsauftrag in kraft gesetzt und mit dem Projektabschluss aufgelöst

Hermes beschreibt 17 verschiedene Rollen

4.2.2. Aufgaben

In HERMES gibt es für jede der ca. 70 Aufgaben eine Aufgabenbeschreibung. Jeder Aufgabe ist eine verantwortliche Rolle zugeordnet. Thematisch zusammengehörende Aufgaben sind in Module gruppiert und den Phasen zugeordnet.

4.2.3. Ergebnisse

Für jedes der ca. 70 Ergebnisse gibt es in HERMES eine Ergebnisbeschreibung. Zudem sind Minimalergebnisse definiert, um die Anforderungen an die Projekt-Governance zu erfüllen (mehr als 40 Ergebnisse).

4.2.4. Sichten auf das Projekt

Sicht des zeitlichen Ablaufs:

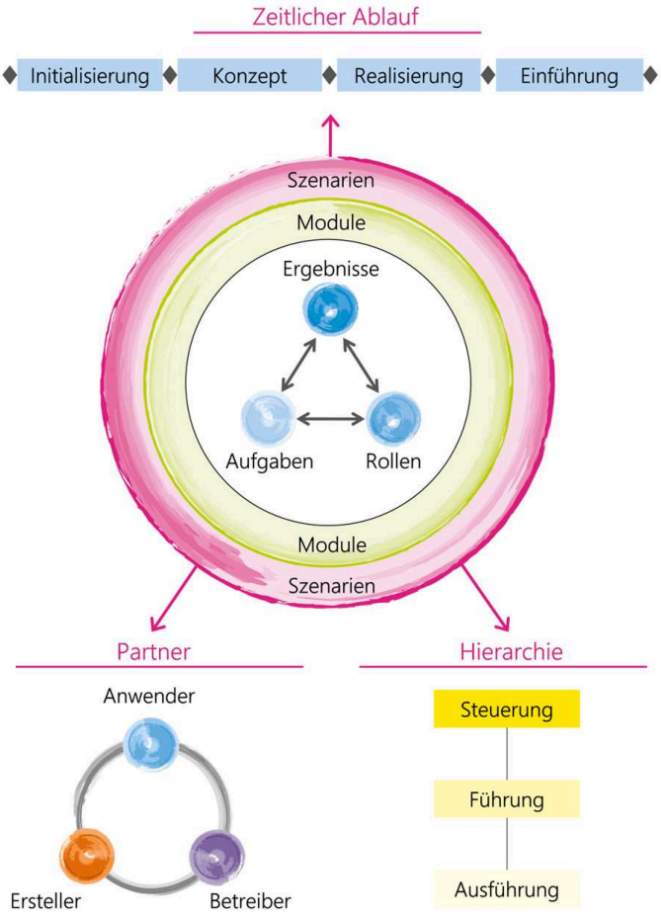
- Welche Aufgaben und Ergebnisse in welcher Phase
- Welche Meilensteine

Sicht der Partner:

- Rollen und Aufgaben im Projekt
- Wo arbeitet diese Rolle mit

Sicht der Hierarchieebenen:

- Entscheidungsaufgaben und Ergebnisse
- Welche Rollen sind der Ebene zugeordnet



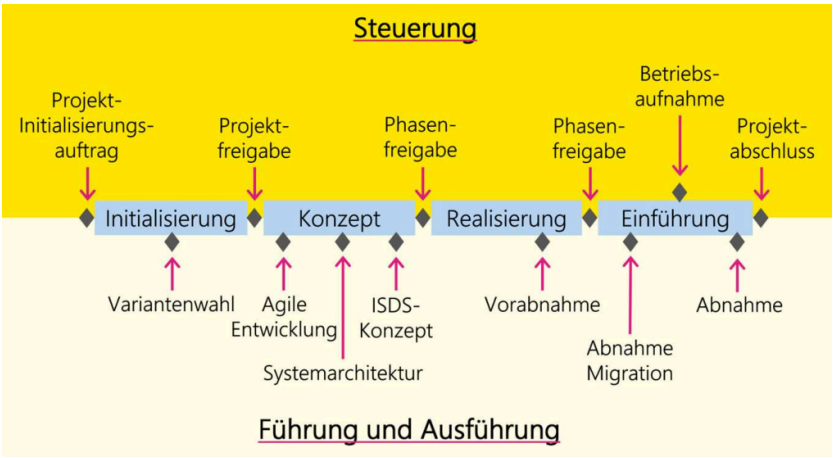
4.2.5. Szenarien

Ein Szenario ist auf die Durchführung eines Projekts mit einer spezifischen Charakteristik ausgerichtet. Die Szenarien bildet den gesamten Lebenszyklus des Projekts ab. Szenarien bilden Module, die thematisch zusammengehörende Aufgaben und Ergebnisse gruppieren.

4.2.6. Entscheidungsprozess

HERMES unterscheidet zwischen:

- Entscheide durch die Steuerung (oben - dunkelgelb)
- Entscheide durch die Projektführung (unten - hellgelb)



4.2.7. HERMES und SCRUM

HERMES deckt den gesamten Lebenszyklus des Projekts ab. SCRUM regelt die Organisation und die Steuerung des Entwicklungsteams.

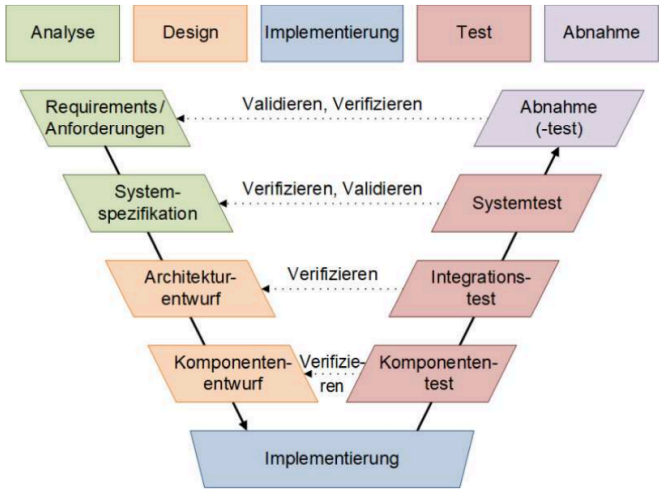
4.2.8. Vor- und Nachteile

Pro	*Contra*
– Hohe Standardisierung mit klar definierten Phasen, Rollen, Aufgaben, Ergebnisse, etc. – Viele Tools und mehrsprachige Vorlagen vorhanden – Eine Zertifizierung erlaubt die Mitarbeit in Bundesprojekten – Einbettung von SCRUM ist klar definiert – Passt zu Institutionen, welche ihre Aufträge öffentlich ausschreiben oder Dienstleister, welche an den Ausschreibungen teilnehmen	– Sehr starke Vorgaben mit wenig Spielraum – Vier Phasen sind etwas knapp bemessen – In der Privatwirtschaft und ausserhalb der Schweiz kaum relevant – HERMES kann Projekte verkomplizieren und zwar so sehr, dass doch einige Bundesprojekte HERMES abspecken

4.3. V-MODELL

Das V-Modell ist ein lineares Vorgehensmodell, das ein Projekt in feste definierte Phasen gliedert. Im Vergleich zum ebenfalls linearen Wasserfallmodell ergänzt es Testphasen, die den jeweiligen Entwicklungsphasen gegenübergestellt sind.

Das V-Modell wird in Deutschland bei Bundesprojekten oft eingesetzt.



4.4. PRINCE2

PRoject IN Controlled Environemnts.

PRINCE2 ist so generisch wie nötig, jedoch so konkret wie möglich, um als ganzheitliche Projektmethodik wahrgenommen zu werden.

Die Methodik gibt die Empfehlungen und Eckpunkte vor, anhand derer der Rhythmus für die Meetings und Hinweise zu deren Inhalt gefunden werden kann.

Wichtigstes Grundprinzip ist: Die Methodik **muss** an die Projektumgebung angepasst werden

PRINCE2 Agile ist eine Erweiterung und Anpassung zum klassischen PRINCE2 für die agile Welt des Projektmanagements.

4.4.1. Vier integrierte Bausteine

1) Sieben Grundprinzipien: oder sieben Leitsätze, die in einem PRINCE2 Projekt eingehalten werden sollen
2) Sieben Themen: Beschreiben den Inhalt eines richtigen Projekts (nach PRINCE2)
3) Sieben Prozesse: Beschreiben den Ablauf rund um die sieben Themen
4) Anpassung an die Projektumgebung: Das Framework kann in weiten Teilen angepasst werden:
   – Vereinfachung der Methodik: Vereinfachung von Techniken und Praktiken
   – Formalisierung bzw. Informalisierung: Meetings werden angepasst - meist informeller

- Umgestaltung von Formaten: Berichte und Tabellen anpassen
- Zusammenführung und Splittung: Berichte adressatengerechter gestalten
- Re-naming: Begrifflichkeiten auf die Organisation anpassen

4.4.2. Sieben Grundprinzipien

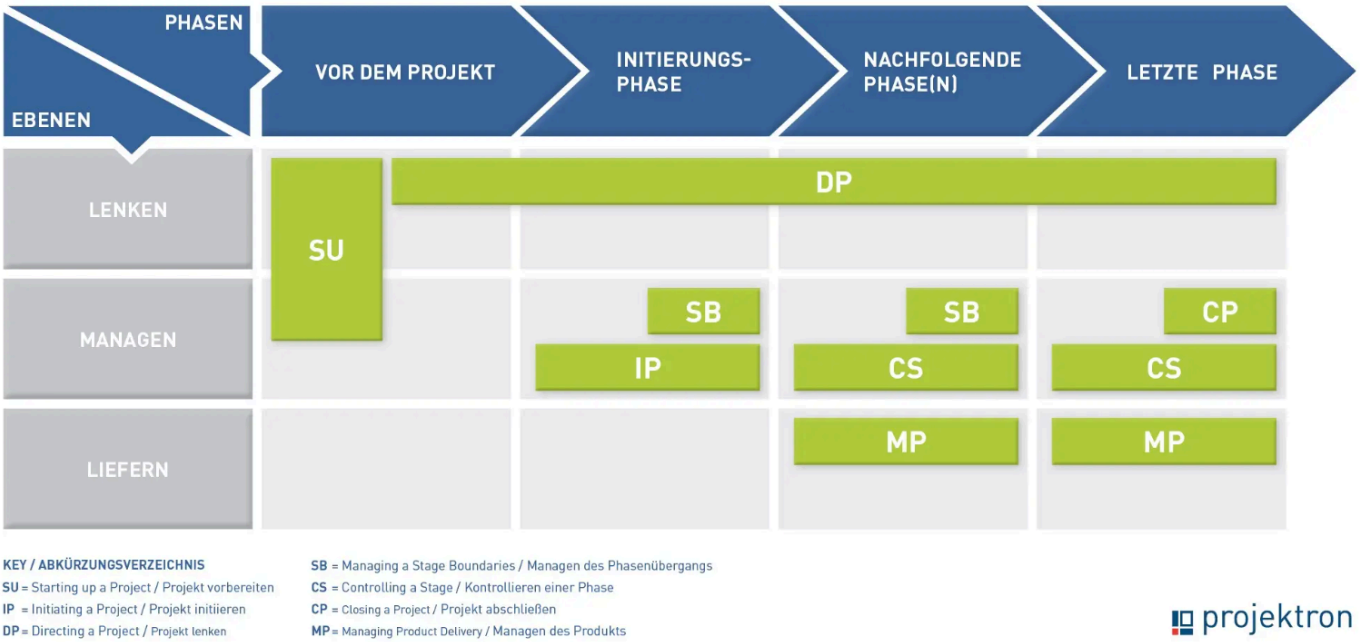
1) **Fortlaufende geschäftliche Rechtfertigung:** Der Nutzen oder Mehrwert muss laufend überprüft werden → Business Case
2) **Lernen aus Erfahrung:** Erfahrungen aus früheren Projekten oder während des Projekts werden regelmässig gesammelt und angewendet.
3) **Definierte Rollen und Verantwortlichkeiten:** Diese müssen auch mit den entsprechenden Kompetenzen und Know How ausgestattet sein.
4) **Steuern über Management Phasen (abgeschlossene, eigenständige Projektphasen):** Am Ende muss der Projektmanager an den Lenkungsausschuss rapportieren.
5) **Steuern nach dem Ausnahmeprinzip (Management by Exception):** Gesteuert wird über das Ergebnis.
6) **Produktorientierung:** Den Blick auf die vom Projekt zu liefernden Produkte lenken
7) **Anpassung an die Projektumgebung:** (siehe Baustein vier)

4.4.3. Sieben Themen

1) **Business Case:** fortlaufende geschäftliche Rechtfertigung. Mechanismus um «geschäftliche Rechtfertigung» zu erlangen und kontinuierlich zu pflegen
2) **Organisation:** definierte Rollen und Verantwortlichkeiten. WER ist für die Umsetzung für WAS verantwortlich?
3) **Qualität:** Produktorientierung. Kundenqualitätserwartungen in Projektabnahmekriterien überführen
4) **Pläne:** steuern über Managementphasen ; Produktorientierung. Wie wird die Planung umgesetzt und wie wird die Umsetzung gemacht?
5) **Risiko:** Umgang mit Unsicherheiten (positive wie negative) in einem Projekt
6) **Änderungen:** Struktur in das Änderungsverfahren und das Konfigurationsmanagement bringen
7) **Fortschritt:** (Reporting, Eskalationen und Toleranzbereich des Projekts). Versetzt das Projektmanagement und den Lenkungsausschuss in die Lage, Entscheidungen zu treffen.

4.4.4. Sieben Prozesse

Die Sieben Prozesse stellen die Ablaufbeschreibung zu den sieben Themen dar. Ein Prozess ist in PRINCE2 wie in der allgemeinen BWL wie folgt definiert: Für einen definierten Input wird über eine vorgeschriebene Abfolge von Aktivitäten Wertschöpfung generiert und ein definierter Output als Mehrwert geliefert.



1) **Vorbereiten eines Projekts:** Starting up a project (SU)
   – Entspricht in etwa einer Vorstudie
   – Oft wird bei grossen Projekten in einer Vorstudie (oder Vorprojekt) die Planung des Grossprojekts erstellt und bestimmt, ob es sich überhaupt lohnt, es durchzuführen (Business Case, Proof of Concept)
2) **Lenken eines Projekts:** Directing a project (DP)
   – Wichtig für den Lenkungsausschuss - damit dieser in jeder Phase die Kontrolle behält
3) **Initiieren eines Projekts:** Initiating a project (IP)
   – Hauptprozess der ersten Projektphase
   – Erstellung des Projektplans und der ersten Arbeitsverteilung
4) **Managen eines Phasenübergangs:** Managing a stage boudary (SB)
   – Definiert den Übergang einer Phase und kommt damit mehrfach identisch vor
5) **Steuern einer Phase:** Controlling a stage (CS)
   – Dieser Prozess ist am umfangreichsten beschrieben, da hier die Hauptarbeit des Projektmanagements liegt
6) **Mangen der Produktlieferung:** Managing product delivery (MP)
   – Arbeitspakete innerhalb und am Ende des Projekts übergeben (innerhalb des Projektteams und zum Kunden)
7) **Abschliessen eines Projekts:** Closing a project (CP)
   – Erfolgreicher Projektabschluss mit allen Dokumenten und Formalitäten sicherstellen

4.4.5. PRINCE2 Agile

– Agile Entwicklungsmethoden wie Kanban, SCUM, Extrem Programing, … stellen keinen ganzheitlichen Projektmanagementansatz dar. Der Fokus liegt auf der Entwicklung eines Produkts.
– PRINCE2 sagt von sich, einen generischen Ansatz zum umfassenden Projektmanagement darzustellen.
– Im Wesentlichen wird nur der Prozess «Managen der Produktlieferung (MP)» durch agile Zyklen erweitert. Die übrigen Prozesse bleiben dieselben.

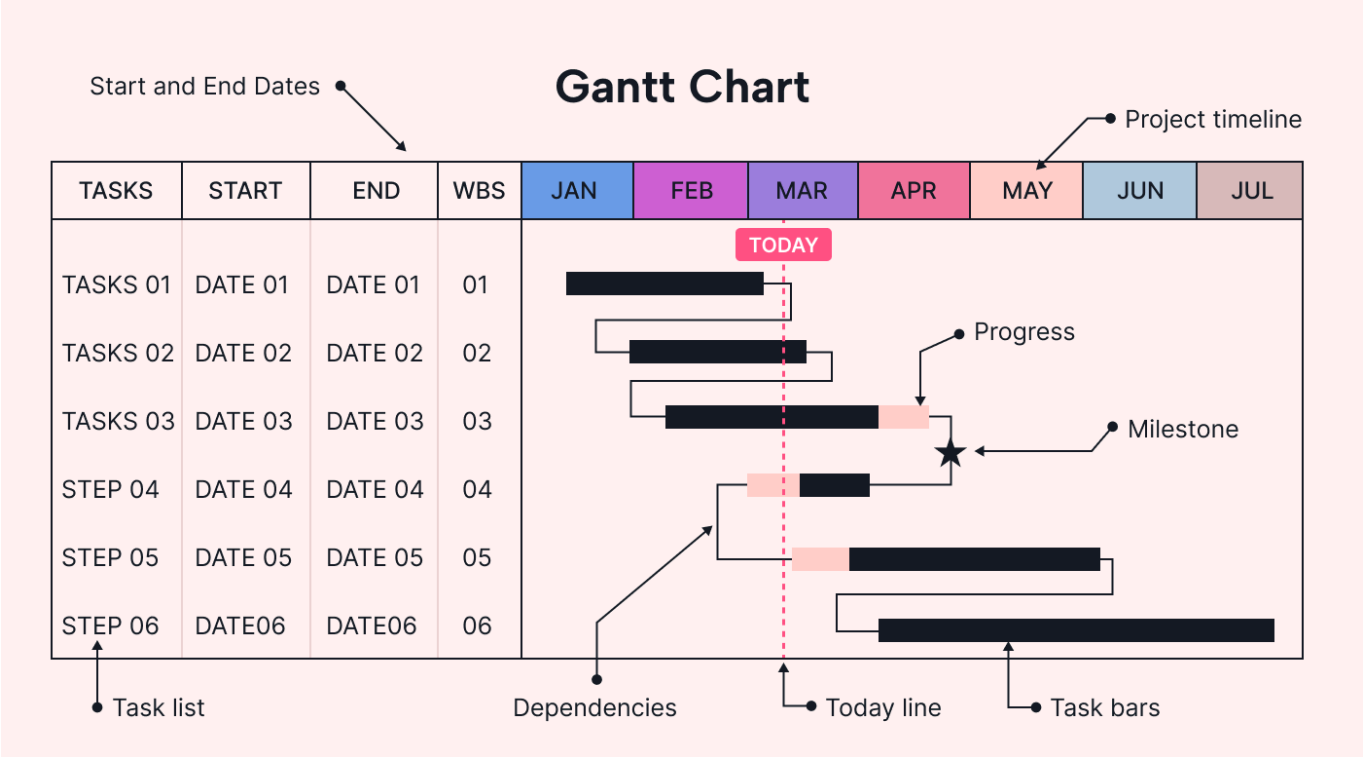
8 Agile Guidance points:

1) **PRINCE2 ist die Basis:** PRINCE2 Agile ist keine neue Version, sondern nur eine Leitlinie, wie die Projektumgebung angepasst werden kann.
2) **PRINCE2 Agile ist neutral zum Entwicklungsprozess:** Damit können alle agilen Entwicklungsmethoden abgedeckt werden.

3) **Agilität kommt aus der IT. PRINCE2 Agile ist nicht IT-limitiert**
4) **Agile Frameworks sind oft IT-limitiert. PRINCE2 Agile nicht**
5) **PRINCE2 Agile ist nicht limitiert auf SCRUM**
6) **SCRUM und Kanban sind die am weitesten verbreiteten Frameworks**
7) **Agilität ist der Zusammenschluss von Behaviors, Konzepten, Framework und Techniken**
8) **Agilität ist nicht binär:** Es stellt sich also nicht die Frage ob agil oder nicht, sondern wie viel agil?

4.5. GANTT

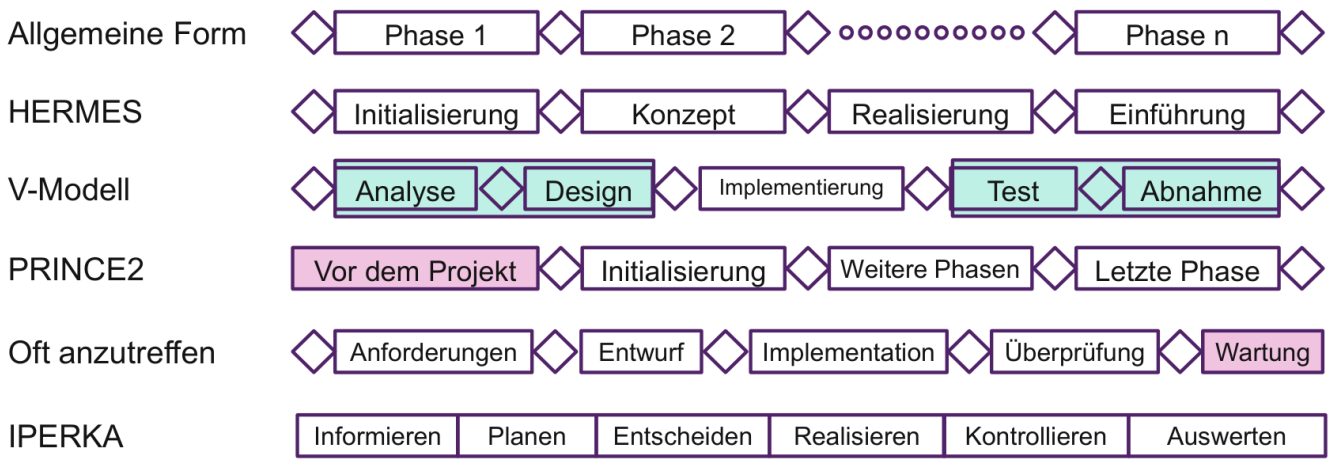
Gantt-Diagramme sind eine der gängigsten und effektivsten Methoden, um Aktivitäten (Aufgaben und Ereignisse) zeitbezogen anzuzeigen.



5. AGILE PRODUKTENTWICKLUNG

Klassische Projektmethode: Material bereitlegen, Planung machen, Struktur erstellen, Produkt erstellen

Agile Produktentwicklung: Erste Möglichkeit zur Nutzung bauen und Kontinuierlich verbessern



5.1. ARBEITSPAKETE

– Alle Aufgaben, die in einem klassischen Projekt ausgeführt werden, müssen in Arbeitspakete gepackt werden.
– Für jedes Arbeitspaket muss jemand zuständig sein.
– Wichtige Arbeitspakete werden von einer zweiten Person überprüft (4-Augen-Prinzip).
– Arbeitspakete sind in sich abgeschlossen und thematisch zusammenhängend.

5.2. WARUM AGILE?

V-Modell: stellt über die Validierung sicher, dass die Anforderungen und Konzepte auf allen Ebenen sauber validiert werden

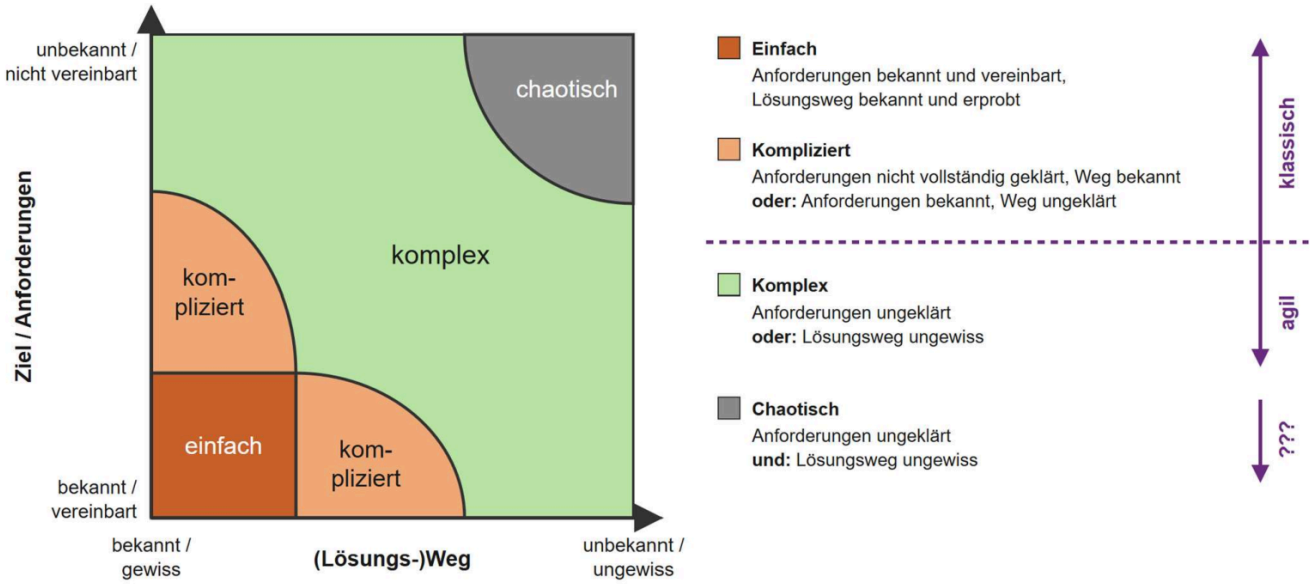
HERMES: Beschreibt alles in allen Details und stellt alle Templates zur Verfügung

PRINCE2: Lässt sich flexibel an alle Umgebungen und Organisationen anpassen

Diese Modelle sind ideal für Werkverträge. Sie ermöglichen ein genau definiertes Ergebnis zu einem definierten Preis an einem vereinbarten Termin.

Im Laufe eines Projekts ergeben sich immer aber Veränderungen.

5.2.1. Stacey-Matrix



TODO:
cynefin

5.2.2. Warum scheitern IT Projekte?

– Schwammige Definition der IT-Projekte
– Planungs- und Methodenfehler
– Projektkommunikation funktioniert nicht
– Fehler bei der Ressourcenzuweisung
– Schlechtes Management von Abnahme und Deployment

5.2.3. Warum werden agile Methoden verwendet

Schneller Start: da zu dessen Beginn nicht alle Details bereits festgelegt und entschieden sein müssen.

Direkter Einfluss des Kunden auf den Projektverlauf: durch die Teilnahme an Meetings, Usability-Tests, Sprint-Planungen und persönliche Abnahme von Sprint- Ergebnissen.

Effektivere Arbeitsabläufe: wenig formeller Overhead

Hohe Flexibilität: gegenüber geänderten Wettbewerbsanforderungen, Kunden- und Nutzerbedürfnissen.

Fehler werden früh erkannt und behoben: damit können Risiken früher erkannt und minimiert werden.

Effektive Ergebnisse: die auf Kunden- und Nutzerbedürfnisse zugeschnitten sind

Schnelle Ergebnisse: und geringe Zeit bis zur Markteinführung (Time-to-Market)

TODO:
agile manifesto

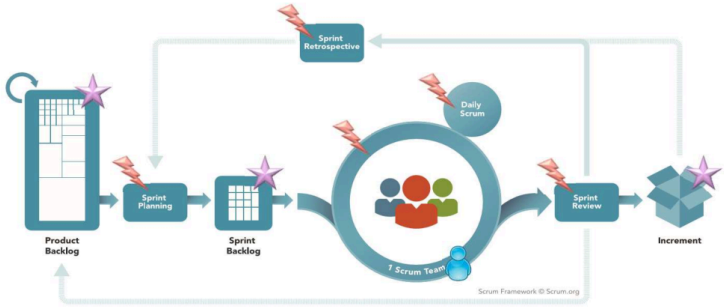
5.3. SCRUM

SCRUM ist ein dynamischer Ansatz und stützt sich auf regelmässiges Feedback von Endbenutzern. Er fördert:
– Mehr Zusammenarbeit mit dem Kunden
– Überwindung der Angst vor Veränderungen
– Interaktion mit Menschen im Mittelpunkt jedes Softwareprojektmanagements
– Fokussierung bei der Lieferung von Software

Rollen
Product Owner
Development Team
Scrum Master

Artefakte
Product Backlog
Sprint Backlog
Increment

Zeremonie	Dauer
Sprint	2 bis 4 Wochen
Sprint Planning	< 4 bis 8 Stunden
Sprint Review	2 bis 4 Stunden
Sprint Retrospektive	1.5 bis 3 Stunden
Daily Scrum	<15 Minuten



Achtung Effizienz:
Die Dauer der Zeremonien müssen aufeinander angepasst sein.

5.3.1. Zeremonien

Sprint Planning: (am Anfang jedes Sprints) Das Team definiert, was im Increment geliefert und wie es gemacht wird, aber der Produkt Owner priorisiert.

Daily Scrum: (täglich während der gesamten Sprint-Dauer) Hier werden Fortschritte besprochen und Blocker identifiziert.

Sprint Review: (Am Ende des Sprints – vor der Auslieferung des Increments) Hier wird besprochen, was während des Sprints erreicht wurde. Dabei werden Stakeholder (und Kunden) und Teammitglieder informiert, die dazu Feedback geben können.

Sprint Retrospective: (nach dem Sprint Review – vor dem nächsten Planning) Hier wird besprochen, in welchen Bereichen während des Sprints Erfolge erzielt wurden und was noch wie verbessert werden kann.

5.3.2. Artefakte

Product Backlog: Beinhaltet alle zurzeit bekannten User Stories und wird vom Produkt Owner in Zusammenarbeit mit dem Kunden und dem Team gepflegt

Sprint Backlog: Beinhaltet die User Stories, an denen in diesem Sprint gearbeitet wird

Increment: Umgesetzte User Stories geprüft und dokumentiert als neue Version. Wird dem Kunden als neue Version zur Verfügung gestellt

5.3.3. Fundament

Transparenz: Der Fortschritt aller Arbeiten und Aufgaben kann vom gesamten Team nachvollzogen werden

Überprüfung: Es können jederzeit Kontrollen durchgeführt und Feedback gegeben werden

Anpassung: Während des gesamten Prozesses können Anpassungen gemacht werden.

5.4. USER STORY UND MVP

Der Product Owner beschreibt in einer User Story die Anforderungen aus Nutzersicht: Wie wünschen sich Anwender eine Software oder Kunden einen Liefergegenstand, um ihre Geschäftsziele zu erreichen. Eine User Story besteht aus wenigen, leicht verständlichen Sätzen. Sie sind kurz, aus Kundensicht, jedoch spezifisch und detailliert.

INVEST-Kriterien (für gute User Stories):

Independent: unabhängig von anderen User Stories

Negotiable: verhandelbar ; diskutierbar

Valuable: nützlich ; muss Wert stiften

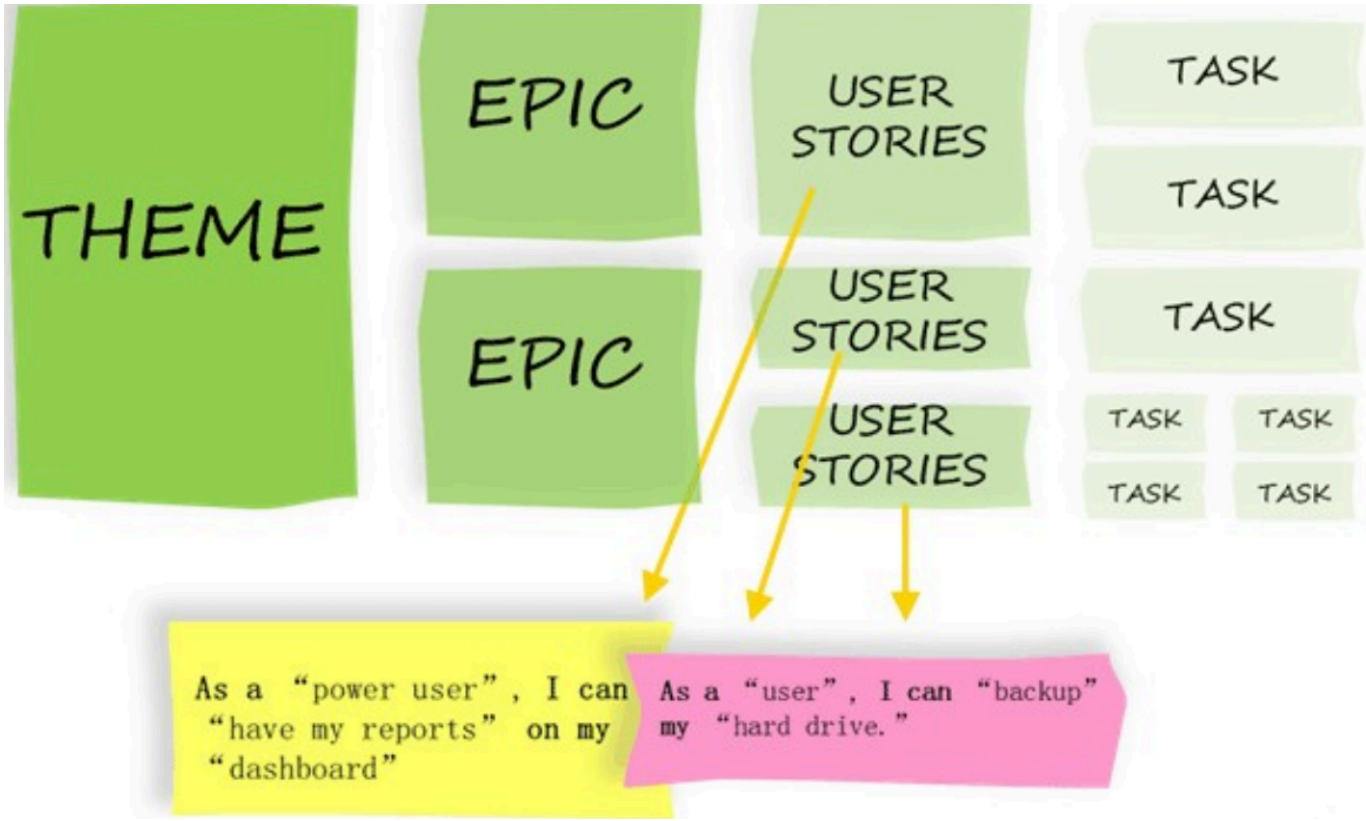
Estimable: Aufwände müssen abschätzbar sein

Small: klein, damit in einem Sprint umsetzbar

Testable: testbar

Eine User Story wird in der agilen Entwicklung anstelle von Anforderungen erstellt und in einem Backlog abgelegt. Schema: «Wer» möchte «was» [und «warum»]. Beispiel: «Als [Rolle] möchte ich [Funktion] um [Nutzen].»

5.4.1. Theme, Epic und User stories



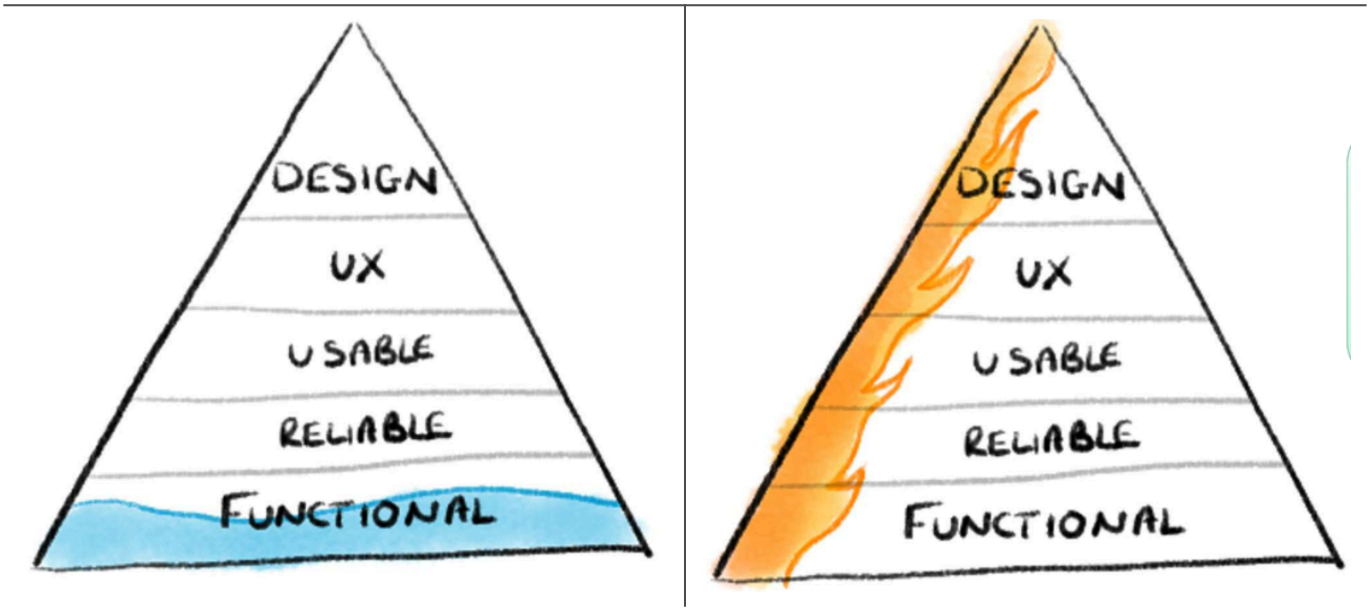
5.4.2. MVP

Das MVP ist «die erste Iteration» eines Produkts oder Services, die dem Nutzer zur Verfügung gestellt wird. Oder es findet noch vorher statt und bildet die Basis für für den Start der Entwicklung.

Auf den freien Markt:

- Passt meine Lösung zum Problem?
- Wer ist mein idealer erster Kunde?
- Wie erreiche ich meine Kunden?
- Was ist der sinnvolle zweite Schritt?
- Software-Entwicklung braucht oft einen längeren Vorlauf, bevor das MVP dem Kunden einen Nutzen bringt und durch Iterationen erweitert werden kann.
- Die Infrastruktur muss soweit bereit stehen, dass die Applikation darauf betrieben werden kann. z.B: Applikations-, Datenbank und Web-Service
- Grundfunktionen müssen implementiert sein. z.B: User Indentifizierung und Autorisierung, Benutzer-oberfläche, …
- Das MVP muss so viel Funktion liefern, dass die Benutzer damit arbeiten können und wollen.
- Das MVP ist der «Erste Eindruck». Ein Produkt hat nur einen «ersten Eindruck», daher muss dieser passen.

Schlechtes MVP	*Gutes MVP*
Bringt zwar Funktionen, ist aber nicht anwendbar. Damit stellt es für den Kunden keinen Nutzen dar. Er wird es ablehnen.	Beinhaltet alle Ebenen und ist damit voll einsetzbar. Der Kunde kann es im vorhandenen(kleinen) Funktionsumfang einsetzen.



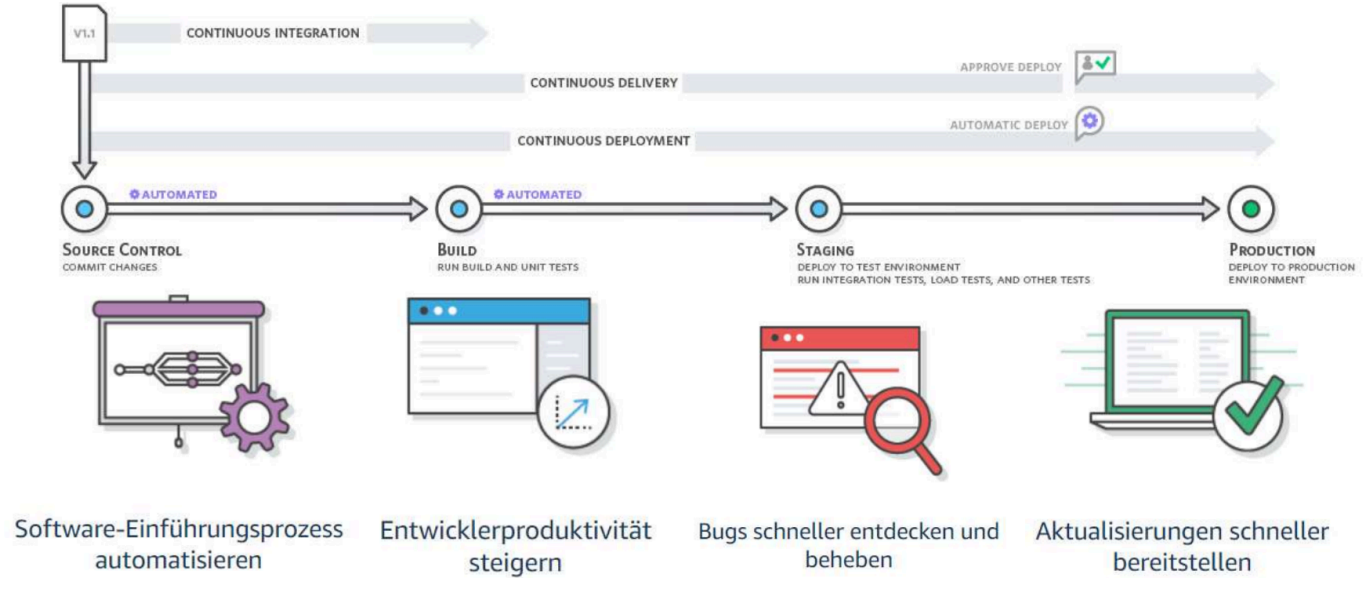
MVP
– Hilft Risiken zu managen
– Ein Produkt zu entwickeln und erfolgreich zu vermarkten ist teuer und braucht Zeit
– Leistungen zu entwickeln, die keiner braucht, verschwendet unnötig Kapital, das wir nicht haben.
– Lange am Produkt zu arbeiten, führt dazu, dass dich Marktteilnehmer überholen.

Das MVP hilft dir kritischen Ressourcen zu managen und Risiken zu senken. Wenn dein MVP scheitert, weisst du, dass es noch nicht die beste Idee war.

5.5. WAS HEISST AGILE

Eine Methode wird als «agil» bezeichnet, wenn darin nach den Prinzipien und Werten des agilen Manifests gearbeitet wird. Das agile Manifest legt den Fokus auf:
1) Kontinuierliche Auslieferung von funktionierender Software in kurzen Zeitabständen
2) Flexibilität bei Änderungen durch Konzentration auf Kundennutzen
3) Zusammenarbeit und Motivation durch Selbstorganisation aller Beteiligten
4) Kontinuierliche Verbesserung und hohe Qualität durch regelmässige Reflexion

5.5.1. (1) CICD



5.5.2. (2) Kundennutzen

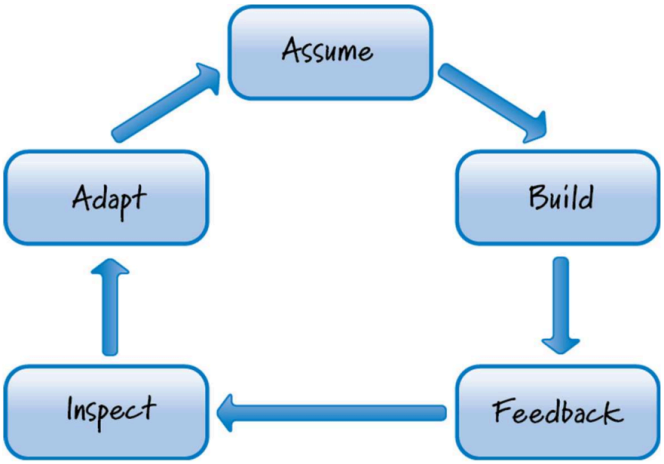
– Priorität auf wertvolle Software für den Kunden
– Änderungswünsche des Kunden jederzeit begrüssen und integrieren
– Je einfacher, desto besser heisst: Alles Unnötige weglassen

5.5.3. (3) Zusammenarbeit

– Informationen stehen jedem zur Verfügung und werden «einfach» gehalten
– Qualität und funktionierende Software gelten als Fortschrittsindikator
– Keine versteckte Agenda oder alternative Fortschrittskontrollen
– Visualisierung der Daten

5.5.4. (4) Inspect and Adapt

– Sämtliche Bestandteile im Produkt werden regelmässig überprüft und wenn notwendig angepasst, d.h. auch wichtige Anforderungen dürfen jederzeit einfliessen.
– Dazu muss regelmässig vom Kunden Feedback eingeholt werden.
– Auch die eigenen Prozesse werden regelmässig hinterfragt und verbessert.



5.6. KANBAN

Entstanden aus dem Toyota Production System.

Diese Themen finden sich auch im Lean Management

In Software:

– Dateneingaben, die nicht nötig sind
– Bestätigungsklicks, die nicht nötig sind

Im Berufsalltag:

– Sitzungen, die nicht nötig sind
– Informationen, die nicht nötig sind
– Arbeitsschritte und Kontrollen, die nicht nötig sind

5.6.1. Nutzen

– Arbeit visualisieren
– Ist-Zustand abbilden
– Workflow beobachten
– Beschränkungen einbauen (Fokus auf aktive Aufgaben)
– Menge der gleichzeitig laufenden Arbeiten (Work in Progress) minimieren
– Gesamtarbeit wird in gut überschaubare Teilaufgaben unterteilt
  – Zu grosse Aufgaben fliessen schlecht durch das System (> 3 Tage)
  – Gute Unterstützung bei der Aufwandsschätzung
– Prognosen ableiten (und reagieren)
– Schnelle durchlaufzeiten

5.6.2. Prinzipien

1) Beginnt mit dem, was ihr ohnehin schon macht
2) Einigt euch auf allmähliche, schrittweise Veränderungen
3) Berücksichtigt bestehende Prozesse, Rollen und Verantwortlichkeiten
4) Fördert Initiativen auf allen Ebenen

Scrum hat diese Flexibilität nicht. Scrum erwartet von Anfang an einen definierten Prozess mit definierten Rollen und Zeremonien in definieren Sprints

5.6.3. Praktiken

1) Visualisiere deine Arbeit
2) Begrenze die Menge gleichzeitig laufender Arbeitsvorgänge
3) Verwalte den Arbeitsfluss
4) Schaffe Klarheit über Prozessregeln
5) Implementiere Feedbackschleifen
6) Verbessere den Prozess gemeinsam und probiere Neues aus

5.6.4. Timebox vs Functionbox

Zeit vor Inhalt	*Inhalt vor Zeit*
– Fixe Zeitdauer, nicht länger und nicht kürzer – Nicht erledigte Arbeiten werden in die nächste Box eingeplant	– Fixer Scope, darf nicht gekürzt oder erweitert werden – Arbeiten werden immer erledigt Das ist aber keine fixe Vorgabe von Kanban. Im Extremfall ist jede User Story eine eigene Iteration und wird direkt released (DI/CD)

5.6.5. Kanban und Scrum

Teams, die mit Scrum arbeiten, nutzen dafür meist dieselben Boards, ebenfalls mit Karten. Aber Scrum ist ein separates Framework, auch wenn Kanban dazu genutzt werden kann. Kanban konzentriert sich auf Prozessverbesserungen. Scrum hilft den Teams, Arbeit schneller zu erledigen. Dafür organisiert Scrum «Sprints» – Arbeitsblöcke mit täglichen Meetings in den Teams und einem festgelegten Arbeitspensum, das während des Sprintzyklus zu erledigen ist.

5.7. WEITERE AGILE METHODEN

5.7.1. XP

– Will bewährte Prinzipien in der Entwicklung bis zum Extremen zu Ende denken.
– Das Ergebnis ist ein leichtgewichtiger Prozess für kleine bis mittelgrosse Teams und Projekte mit unklaren oder sich stark ändernden Anforderungen.
– XP basiert auf den Werten Kommunikation, Einfachheit, Feedback, Mut und Respekt.

5.7.1.1. Wichtigste Grundsätze

Zusammensitzen: Entwicklung soll im selben Raum erfolgen

Das ganze Team: Alle benötigten Personen sollen ins Team einbezogen sein

Informative Arbeitsumgebung: soll die Arbeit widerspiegeln

Energiegeladene Arbeit: nur so lange arbeiten, wie man das produktiv und dauerhaft tun kann

Pair Programming: Alle Programmteile werden von zwei Entwicklern am selben Computer gemeinsam entwickelt

Stories: Funktionseinheiten werden in Stories beschrieben, die für den Kunden sichtbar und verständlich sind

Wöchentlicher Zyklus: Arbeit und Umsetzung wird auf wöchentlicher Basis geplant

Quartalszyklus: einmal im Quartal wird über das Team, das Projekt, den Fortschritt und die Ziele nachgedacht

Entspannung: In jedem Plan werden Aufgaben eingeplant, die fals zeitlich nötig auch weggelassen werden können

Zehn-Minuten-Builds: Das System soll in zehn Minuten inklusive Tests erstellt werden können

Fortlaufende Integration: Abgeschlossene Aufgaben werden immer direkt ins System integriert

Test-First: Tests werden jeweils vor der Implementierung geschrieben

Inkrementelles Design: Das Design soll täglich den Erfordernissen angepasst werden

5.7.2. Scaled Agile Framework / Agilität im Multiteam

Das Scaled Agile Framework (SAFe) ist das verbreitetste Rahmenwerk, um Scrum bzw. Agil zu skalieren - sprich, im «Grossen» zu leben. SAFe bietet agile Lösungen für alle Ebenen einer Organisation: von Teams über Bereiche bis hin zur Unternehmenssteuerung.

5.7.2.1. Problem: übergreifende Funktionen

Ein Team reicht heute selten aus, um ein Produkt zu entwickeln. Mehrere Teams müssen koordiniert auf das Gesamtprodukt hin entwickeln. Dazu müssen die Teams passend geschnitten werden: Story Mapping, Customer Journey (Kundenerlebnis), Domain Driven Design, etc.

Kann mit Verwaltung gelöst werden (führt aber zu Trägheit und schlechter Vorhersagbarkeit).
– Aufgaben werden an die Team-Struktur angepasst
– Team-Struktur wird an die Aufgaben angepasst (leider nur selten)

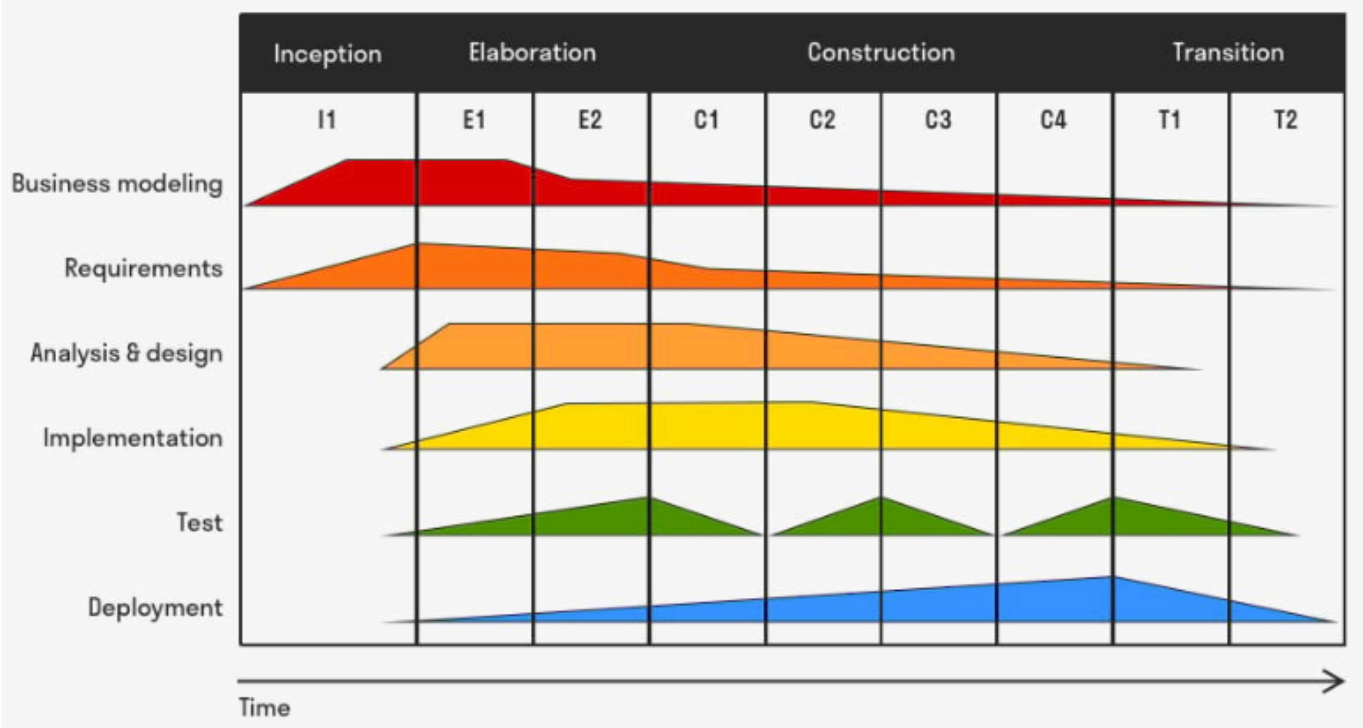
Feature Manager: Übergreifende Funktionalitäten werden in Teilfunktionen je Team geschnitten. Ein Feature Manager definiert die Aufgabenpakete für die einzelnen Teams und kontrolliert den Fortschritt. (Delivery Manager/ Chief Product Owner/ Program Manage/ Epic Owner)

Big Room Planning: setzt auf die Eigenverantwortlichkeit der Teams. Es finden sich alle Teammitglieder aller Teams für eine gemeinsame Planung zusammen. Das kann eine gemeinsame Planung über einen längeren Zeitraum sein (wie z.B. das PI2)-Planning in SAFe).

Teamzuordnung:
– Ein Team übernimmt die Entwicklung der übergeordneten Funktion.
– Bedingt, dass die verschiedenen Teams möglichst homogene Entwicklungstools und Methoden einsetzen.

Temporäres Team:
– Es wird temporär ein Team aus Members der anderen Teams zusammengestellt, das die Entwicklung der übergeordneten Funktion übernimmt.
– Ergibt eine dynamische Teamzusammenstellung und Unruhe in den Teams.

6. RATIONAL UNIFIED PROCESS (RUP)



RUP	*Wasserfall*
– Phasen: Inception, Elaboration, Construction, Transition – Iterationen in jeder Phase – Zunehmend detailliertere Planung	– Phasen abhängig vom Modell – Detaillierte Planung am Anfang über alle Phasen – Phase ist abgeschlossen, wenn alles erledigt – Keine Iterationen, kein zurückspringen

6.1. ALLGEMEIN

RUP ist ein Softwareentwicklungsprozess (und keine eigentlichen Projektmanagement Methode)

6.1.1. Vorteile

– RUP orientiert sich im Wesentlichen an Prinzipien
– Neuste Vorgehensweisen (neben Altbewährtem)
– Grosse Verbreitung (IBM und weitere grosse Beratungsfirmen)
– Detaillierte und klare Beschreibung
– Inhalt und Dokumentation in HTML festgehalten
– Anpassbarkeit des Prozesses (oder besser: muss angepasst werden) z.B. mit IBM Rational Methode Composer

6.2. PRINZIPIEN

Prozess anpassen:

– Prozess muss an das Projekt angepasst werden
– Erhöht die Effizienz, die Agilität und führt zu realistischeren Plänen und Schätzungen
– Wie viel Prozess ist notwendig?

← Weniger prozesse | *Mehr prozesse →*

– Einfache Upgrades – Entwicklungsprototypen – Statische Webanwendungen	– Dynamische Webanwendungen – Komponentenbasierte Entwicklung (J2, .NET)	– Legacy Upgrades – Systemfamilien – Realtime Enbedded – Zertifizierte Qualität
– Teams am selben Ort – Kleine, einfache Projekte – Wenige Stakeholder – Interne Rahmenbedingungen		– Verteilte Teams – Grosse Projekte – Viele Stakeholder – Externe Standards, Gesetze, Verträge

Konkurrenzierende Stakeholder-Prioritäten ausgleichen:

– Unterschiedliche, sich widersprechende Bedürfnisse / Anforderungen müssen geklärt werden. was sind die Interessen des Geschäfts und der Anwender?
– Anforderungen müssen aber am Anfang noch nicht detailliert aufgenommen werden. Sie werden sich im Laufe der Entwicklung ändern!

Teamübergreifend zusammenarbeiten:

– Verbessert die Produktivität, erhöht die Wahrung der Geschäftsinteressen und verbessert die Kooperation mit dem produktiven Betrieb eines Systems.

Wert iterativ demonstrieren:

– Ermöglicht frühes Reduzieren von Risiken, verbessert die Vorhersehbarkeit im Projektverlauf und erhöht das Vertrauen aller Projektbeteiligten.

Abstraktionsgrad erhöhen:

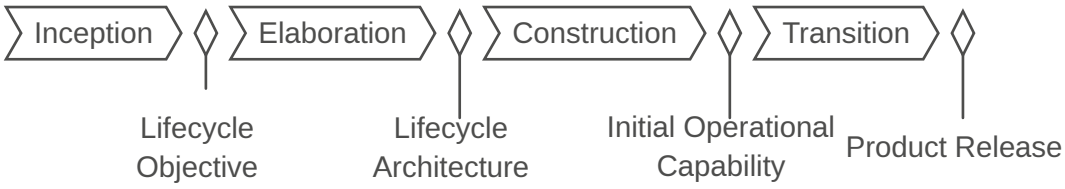
– Damit vermindert sich die Komplexität und die Produktivität kann gesteigert werden.
– Lösungen können wieder verwendet werden.
– Es werden abstraktere Tools und höhere Sprachen eingesetzt.
– Der Fokus liegt auf der Architektur.
– Zur Visualisierung, Spezifikation, Konstruktion und Dokumentation wird ULM (Unified Modelling Language) eingesetzt.

Ständige Konzentration auf Qualität:

– Qualität wird nur erreicht, wenn sie während der ganzen Lebensdauer angestrebt wird.
– Das ganze Team ist für die Qualität des Produkts verantwortlich.
– Durch die ständigen Tests in den Iterationen wird immer auf die Qualität fokussiert.

6.3. PHASEN

RUP kennt vier Phasen. Jede Phase hat unterschiedliche Schwerpunkte. Die Aufgeben innerhalb einer Phase müssen nicht abgeschlossen sein, um in die nächste Phase zu wechseln. Der Phasenplan wird in der Inception Phase in einer ersten Version erarbeitet und später konkretisiert.



Inception ; Konzeption:

– Ziele und Vision des Projekts bestimmen
– Erste Risikoabschätzung

- Use Cases des Systems grob festhalten (nicht ausformuliert, da sie sicher noch ändern)
- Abschätzung von Kosten und Nutzen
- Grobe Projektplanung

Elaboration ; Ausarbeitung:

- Anforderungen stabilisieren
- Höchste Risiken minimieren
- Use Cases und Anforderungen zu 80% ausarbeiten
- Architektur definieren ; Architekturprototyp festlegen
  - Dieser wird während der kommenden Phasen kontinuierlich weiterentwickelt, bis zu fertigem Produkt

Construction ; Erstellung:

- Entwickeln und Testen der Systemkomponenten basierend auf der Architektur
- Am Ende dieser Phase ist ein einsetzbares Produkt

Transition ; Überführung:

- Übergabe an den Kunden
- Abnahme Tests (Beta Tests) und Fehlerbehebung
- Schulung der User
- Fertigstellung des Begleitmaterials
- Ev. Konvertierung der bestehenden Daten

6.4. ITERATIONEN - VIER REGELN

1) Inkrementell
   - Ergebnisse einer Iteration sollen den Projektbeteiligten, den Kunden und den Anwendern gezeigt werden
   - Feedback einholen und auf Fehlentwicklungen aufmerksam machen
2) Pläne anpassen
   - Nach jedem Inkrement sollen die Pläne angepasst und die Anforderungen überprüft werden
3) Änderungen behandeln und verwalten
   - Änderungen sind in Software Projekten sind eher die Regel als die Ausnahme
4) Die grössten Risiken früh ausräumen
   - Risiko Liste erstellen und bei jeder Iteration überarbeiten
   - Risiken in der Phase Elaboration ausräumen (PoC’s)

6.5. DISZIPLINEN

Business Modelling:

- Analyse der gegenwärtigen Organisation und Festlegung der Geschäftsprozessziele
- Pflege des fachlichen Wörterbuchs (Glossar) und der Business Rules
- Business Use Cases (Akteure und Aufgaben zur Erfüllung der Businessziele)
- Definition der Automatisierungen, die möglich sind

Requirements: (über die ganze Lebensdauer des Projekts)

- Anforderungsmanagementplan und Vision entwickeln
- Anforderungen und Wünsche der Stakeholder ermitteln
- Akteure und Use Cases bestimmen
- Use Cases beschreiben und Prioritäten vergeben
- Zusätzliche Anforderungen aufnehmen, die keinem Use Case zugeordnet werden können

Analyse und Design:

- Architektur Machbarkeit
- Organisation der wichtigsten Designelemente inkl. Bestehende integrieren
- Implementierungsmodell identifizieren und strukturieren (Themen, die in der SW immer wieder gleich behandelt werden sollen. Z.B. Fehlerbehandlung, Logging, …)
- Laufzeitarchitektur und Verteilung auf Rechnerknoten bestimmen
- Services für Serviceorientierte Architektur bestimmen
- Designs von Subsystemen, Klassen, Services und Datenbanken

Implementation:

- Implementierungsmodell (Softwarearchitekt)
- Designelemente implementieren, Entwicklertests erstellen und durchführen und Laufzeitverhalten analysieren (Implementierter)
- Subsystemintegration (Integrator)
- Code prüfen (Technischer Prüfer)

Test:

- Identifizieren der Anforderungen, Zielen und Lieferobjekte der Tests in dieser Iteration
- Definition der Qualitätsansprüche an die Tests
- Beschreibung der konkreten Testtechniken und der Hard- und Softwareumgebung
- Durchführen der Tests (automatisch oder manuell nach Testdrehbuch)

Deployment:

- Das Deployment kann in Form einer kundenspezifischen Installation, verteilen eines Standardprodukts oder über einen Zugang zu einer existierenden Umgebung (SaaS) erfolgen
- Deployment Plan erstellen
- Beta-Tests und Akzeptanztests managen
- Lieferliste (Stückliste) erstellen und Fertigungsfreigabe erteilen (für Massenverteilungen) oder Zugriff auf Downloadseite ermöglichen
- Fertiges Produkt verifizieren und Releasenotes schreiben
- Unterstützungs- und Trainingsmaterial bereitstellen und User (Kunden) informieren

Configutation- und Change-Management:

- Erstellen der Konfigurationsmanagement Umgebung (z.B. GIT)
- Definieren des Change-Prozess
- Baseline der Software in einem eigenen Workspace integrieren und für die weitere Verwendung dokumentieren

Environment:

- Entwicklungsprozess für das Projekt anpassen ; Development Case (Wann, wieviel von was)
- Entwicklungsprozess einführen und Team darin schulen (vergleichbar mit SCUM Master)
- Tool zur Projektunterstützung evaluieren und einführen (inkl. Richtlinien für den Umgang damit)

Projektmanagement:

- Planen und Steuern des Projekts
  - Projektstart
  - Entwicklungsplanung
  - Iterationsplanung wie Start und Ende
  - Riskmanagement
  - Quality Assurance,
- Sicherstellen, dass vorgegebene Rahmenbedingungen eingehalten werden und wo nötig Gegensteuer zu geben
  - Maximalbudget
  - Spätester Auslieferungszeitpunkt

  - Funktionalität
- Projektstatus prüfen und Reviews durchführen
- Stakeholder Management (Informationen)

6.6. ITERATIVE PROJEKTPLANUNG

Wesentlicher Unterschied zu herkömmlichen Wasserfallprojekten:
- Die Anforderungen sind nicht von Anfang an bekannt.
- Es müssen viele Risiken (insbesondere technische) erwartet werden.

Es werden zwei Arten von Planungen gemacht:

Projektplanung (Projekt Schedule): grober Zeitplan über die Phasen des Projekts (Interseption, Elaboration, Construction, Transition) Dieser Plan wird situativ abgepasst

Iterationsplanung: Entspricht im Kleinen am ehesten einem Wasserfallprojekt (oder einer Sprintplanung).
- Es wird jedoch nur gerade die aktuelle Iteration geplant.
- Für jede Iteration werden Erfolgskriterien festgelegt und am Ende der Projektfortschritt damit geprüft.

6.7. TYPISCHE FEHLER

Rückfall in Wasserfallpraktiken:
- Die vier Phasen des RUP werden mit den Phasen des Wasserfalls verwechselt
- Dabei wird versucht, schon am Anfang das ganze Projekt im Detail zu planen.
- Häufig wird dabei das Ziel der „ausführbaren Software“ aus den Augen verloren.

Verwendung des RUP ohne Anpassung:
- RUP muss zwingend dem aktuellen Projekt angepasst werden.
- Für ein kleines Projekt können nicht dieselben Lieferergebnisse wie bei einem grossen Projekt erwartet werden.

Transformation der Entwicklungsmethode ohne Berücksichtigung des Unternehmens:
- Die Umstellung auf agile Projektmethoden hat einen Einfluss auf die Unternehmenskultur (siehe Agiles Manifest)

7. IT-SERVICEMANAGEMENT

- Unternehmen sind zunehmend von der IT abhängig ⇒ wachsender Bedarf an IT Services
- Information sind das wertvollste Gut einer Unternehmung
  - Sicherheit (Vertraulichkeit, Verfügbarkeit, Richtigkeit)
  - Cyber Crime wird heute als grösstes Risiko eingeschätzt
- Trend weg von der Entwicklung der IT Anwendungen hin zum Management von IT Services (SaaS)
  - Eine Anwendung unterstützt erst dann die Businessziele, wenn sie voll einsetzbar ist (fit for purpose ; fit for use)
    Betrieb, Support, Wartung, Anpassungen

7.1. ZIELE

- Zuverlässige und konsistente IT Services
  - mit hoher Qualität
  - zu tragbaren Kosten
- Abstimmung der IT Services auf die Geschäftsanforderungen

  - Kostenoptimierung
  - Qualitätssicherung (steuerbares Qualitätsniveau)
  - Betriebsoptimierung
- Trennung von Entwicklung und Betrieb
  - neuer Trend: Dev-Ops / AI-Ops
- Definierte Schnittstelle zum Business (Kunde / Benutzer)
  - neuer Trend: Agile IT ; Produkte-Teams

7.2. BUSINESS NUTZEN

Return on Investment (ROI) aus Sicht des Kunden: Der Kunde will seine Kosten möglichst bald durch den zusätzlichen Nutzen kompensieren können

In einer sehr einfachen Provider-Consumer-Beziehung stellt ein Provider einen Service zur Verfügung, der von einem Consumer angewendet wird.

Grössere Organisationen setzen eine partnerschaftliche Beziehung zwischen Provider und Consumer voraus.

Die Utility wie auch die Warranty müssen erfüllt sein, damit der gewünschte Nutzen entstehen kann. Die Utility wird oft sehr genau in einem Pflichtenheft oder einer Service Vereinbarung festgelegt. Dabei kommt der Warrantry oft zu wenig Beachtung zu.

7.3. VIER SERVICE MANAGEMENT DIMENSIONEN

Organisation	*Information*	*Partner & Supplier*	*Value Streams*
Struktur	Technologie	Beziehungen	Prozesse
Hierachie	Knowledge	Verträge	Aktivitäten
Kultur	Inputs	Vereinbarungen	Workflows
Kapazität	Outputs		Controls
Kompetenzen	Relationship		Prozeduren
Skills			
Management			
Leadership Style			
Kommunikation			
Zusammenarbeit			

7.4. SERVICE VALUE SYSTEM

Kreiert aus Ressourcen Value für den Kunden, um einen Demand zu befriedigen.



- Die grösste Herausforderung einer Organisation in einem agilen Umfeld sind Silos.

– Diese sind oft «change resistent» und verhindern den schnellen Zugriff auf Informationen und Spezialisten.
– Solche Silos können auch durch Prozesse oder Praktiken (Practices) entstehen, wenn diese nicht genügend flexible Schnittstellen aufweisen.
– Silos reduzieren die Effizienz und erhöhen die Kosten und die Risiken.
– Silos erhöhen die Komplexität der Organisation.

Komplexität ist ein Service- und Qualiäts-Killer und Kostentreiber

– ITIL gibt kein fixes System von Value Streams vor.
– Organisationen müssen in der Lage sein, diese selbst zu gestalten und regelmässig und schnell den aktuellen Anforderungen anzupassen.
– Die umfassenden Operationen sind in ITIL definiert durch leitende Prinzipien (guiding Principals) beschrieben.

TODO:
W5